

Smallest Rectangle Enclosing Black Pixels

Given a binary image.

'0' = White pixel

'1' = Black pixel

You are also given one black pixel (x, y) .

All black pixels belong to one connected component.

Find the area of the smallest rectangle containing all black pixels.

Example Observation

Suppose Aadya draws this image.

0	0	1	0
0	1	1	0
0	1	0	0

Given

$x = 0$

$y = 2$

We already know this is a black pixel.

The black pixels are:

- $(0, 2)$
- $(1, 1)$
- $(1, 2)$
- $(2, 1)$

The Enclosing Rectangle

	minCol=1 ↓ maxCol=2 ↓	
0	[0]	1]
1	[1]	1]
2	[1]	0]

Rows

minRow = 0

maxRow = 2

Columns

minCol = 1

maxCol = 2

Area

height = 2 - 0 + 1 = 3

width = 2 - 1 + 1 = 2

Answer = 6

Approach 1: Depth First Search (DFS)

During DFS, just keep updating *minRow*, *maxRow*, *minCol*, *maxCol*.

Visual DFS Update Logic

minRow = min(minRow, r) | *maxRow* = max(maxRow, r)

minCol = min(minCol, c) | *maxCol* = max(maxCol, c)

```

class Solution {
public:
    vector<pair<int,int>> dir={{1,0},{-1,0},{0,1},{0,-1}};
    int n, m;
    int minRow, maxRow, minCol, maxCol;

    void dfs(int r, int c, vector<vector<char>>& image) {
        image[r][c] = '0';

        minRow = min(minRow, r);
        maxRow = max(maxRow, r);
        minCol = min(minCol, c);
        maxCol = max(maxCol, c);

        for(auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;

            if(nr >= 0 && nr < n && nc >= 0 && nc < m) {
                if(image[nr][nc] == '1') {
                    dfs(nr, nc, image);
                }
            }
        }
    }

    int minArea(vector<vector<char>>& image, int x, int y) {
        n = image.size();
        m = image[0].size();

        minRow = maxRow = x;
        minCol = maxCol = y;

        dfs(x, y, image);

        return (maxRow - minRow + 1) * (maxCol - minCol + 1);
    }
};

```

Complexity Analysis (DFS)

DFS visits each black pixel once.

Time: $O(K)$

K = Number of black pixels

Space: $O(K)$ (recursion stack)

The Important Observation

Suppose Aadya draws:

0	0	0	0	0
0	0	1	1	0
0	1	1	0	0
0	0	1	0	0
0	0	0	0	0

We know one black pixel is $(x, y) = (2, 2)$.

Let's only look at rows.

Row 0 -> 00000	NO
Row 1 -> 00110	YES
Row 2 -> 01100	YES
Row 3 -> 00100	YES
Row 4 -> 00000	NO

Does this row contain any black pixel?

Interesting. The YES rows are continuous.

NO	NO	NO
YES	YES	YES
NO	NO	

There cannot be:

YES

NO

YES

because all black pixels belong to one connected component.

Same for columns

Column 0 -> 00000 (vertically)	NO
Column 1 -> 00100	YES
Column 2 -> 01110	YES
Column 3 -> 01000	YES
Column 4 -> 00000	NO

Again,
NO
YES YES YES
NO
continuous.

This is the monotonic property.

Rows look like: False False False | True True True | False False

Columns also look like: False | True True | False

So we can binary search.

Binary Search Logic

What do we binary search? We need:

- top row
- bottom row
- left column
- right column

So we perform 4 binary searches.

Finding Top Boundary

We know Aadya is at row **x**.

Everything above **x** may or may not contain black pixels.

Search range: **0 ... x**

Predicate: *Does this row contain a black pixel?*

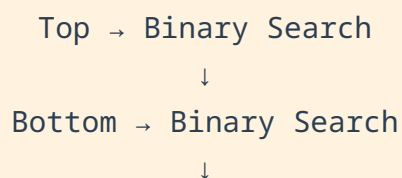
Suppose :

F F F T T T
 ^

We want: **First True**

Classic binary search.

Complete Algorithm Flow



Left → Binary Search



Right → Binary Search



Area

Complexity (Binary Search)

Each row check: $O(m)$

Binary search performs $\log n$ iterations.

So:

Top: $O(m \log n)$ | Bottom: $O(m \log n)$

Similarly:

Left: $O(n \log m)$ | Right: $O(n \log m)$

Total Time: $O(m \log n + n \log m)$

Optimal Approach Code

```

class Solution {
public:
    bool rowHasBlack(vector<vector<char>>& image, int row) {
        int m = image[0].size();
        for(int j = 0; j < m; j++) {
            if(image[row][j] == '1') {
                return true;
            }
        }
        return false;
    }

    bool colHasBlack(vector<vector<char>>& image, int col) {
        int n = image.size();
        for(int i = 0; i < n; i++) {
            if(image[i][col] == '1') {
                return true;
            }
        }
        return false;
    }

    int minArea(vector<vector<char>>& image, int x, int y) {
        int n = image.size();
        int m = image[0].size();

        int topAns, bottomAns, leftAns, rightAns;

        //----- Top Boundary -----
        int l = 0;
        int h = x;
        while(l <= h) {
            int mid = l + (h - l) / 2;
            if(rowHasBlack(image, mid)) {
                topAns = mid;
                h = mid - 1;
            } else {
                l = mid + 1;
            }
        }

        //----- Bottom Boundary -----
        l = x;
        h = n - 1;
        while(l <= h) {
            int mid = l + (h - l) / 2;
            if(rowHasBlack(image, mid)) {
                bottomAns = mid;
                l = mid + 1;
            } else {
                h = mid - 1;
            }
        }
    }
}

```

```

//----- Left Boundary -----
l = 0;
h = y;
while(l <= h) {
    int mid = l + (h - l) / 2;
    if(colHasBlack(image, mid)) {
        leftAns = mid;
        h = mid - 1;
    } else {
        l = mid + 1;
    }
}

//----- Right Boundary -----
l = y;
h = m - 1;
while(l <= h) {
    int mid = l + (h - l) / 2;
    if(colHasBlack(image, mid)) {
        rightAns = mid;
        l = mid + 1;
    } else {
        h = mid - 1;
    }
}

return (bottomAns - topAns + 1) * (rightAns - leftAns + 1);
}
};

```